Matriz de trazabilidad de RentaMovil

Proyecto formativo RentaMovil · Equipo 07 · Ficha 3145556 · Actualizada el 15 de octubre de 2026

1. Propósito y alcance

Esta matriz relaciona cada requisito funcional (RF) con su historia de usuario (HU), el módulo que lo implementa (servicio, endpoint), las pruebas automatizadas que lo verifican y el repositorio donde se encuentra el cambio. Se elaboró revisando el código de los repositorios del proyecto a la fecha indicada. Los requisitos y las historias provienen de `rtm-docs/04-requirements` (`user-stories.md` y `traceability-matrix.md`).

La matriz original (`traceability-matrix.md`) se escribió en la fase de documentación, con nombres de pruebas y de servicios planificados. Esta versión la reemplaza con lo que existe hoy en el código.

Estado: ✅ implementado con pruebas automatizadas · ⚠️ implementado sin una prueba específica identificada · ❌ no implementado.

2. Requisitos funcionales → historia → implementación → pruebas

RF	Descripción	Historia(s)	Implementación (servicio · endpoint)	Pruebas automatizadas	Estado
RF1.1	Registro de cliente	HU-IAM-001	iam · `POST /register`	`RegisterUserUseCaseImplTest`	✅
RF1.2	Inicio de sesión	HU-IAM-002	iam · `POST /login`, `/refresh`, `/logout`	`LoginUseCaseImplTest`, `RefreshTokenUseCaseImplTest`, `LogoutUseCaseImplTest`	✅
RF1.3	Recuperación de contraseña	HU-IAM-003	iam · `POST /password/forgot`, `/password/verify-code`, `/password/reset`	`ResetPasswordUseCaseImplTest`	⚠️ solo se prueba el restablecimiento
RF1.4	Gestión de datos de la cuenta	HU-IAM-004	iam · `GET/PATCH /users/me`, `PATCH /users/me/email`, `PATCH /users/me/password`	`UpdateMyProfileUseCaseImplTest`, `ChangeMyEmailUseCaseImplTest`, `ChangeMyPasswordUseCaseImplTest`	✅
RF8.1	Gestión administrativa de usuarios	HU-IAM-005	iam · `GET /users`, `PATCH /users/{id}/role`, `PATCH /users/{id}/status`	`ChangeUserRoleUseCaseImplTest`, `ChangeUserStatusUseCaseImplTest`	✅
RF2.1	Consultar y filtrar el catálogo de vehículos	HU-FLEET-001, HU-FLEET-003	fleet-maintenance · `GET /vehicles`, `/vehicles/inventory`, `/vehicles/stats`	`SearchVehiclesUseCaseImplTest`	✅
RF2.2	Consultar la disponibilidad de un vehículo	HU-FLEET-002	fleet-maintenance · `GET /vehicles/{id}/availability` (usa `booking-reservation · GET /internal/reservations/vehicle-occupancy`)	`CheckVehicleAvailabilityUseCaseImplTest`	✅
RF8.2	Registrar, editar y desactivar vehículos	HU-FLEET-004, HU-FLEET-005	fleet-maintenance · `POST /vehicles`, `PUT /vehicles/{id}`	`RegisterVehicleUseCaseImplTest`, `UpdateVehicleUseCaseImplTest`	✅

RF	Descripción	Historia(s)	Implementación (servicio · endpoint)	Pruebas automatizadas	Estado
RF8.3	Gestión del estado del vehículo	HU-FLEET-006	fleet-maintenance · `PATCH /vehicles/{id}/status`, `POST /internal/events`	`ChangeVehicleStatusUseCaseImplTest`, `HandleRentalEventUseCaseImplTest`	✅
RF8.4	Gestión de sucursales	HU-FLEET-007	fleet-maintenance · `GET/POST /branches`, `PUT/DELETE /branches/{id}`	`ManageBranchUseCaseImplTest`	✅
RF8.7	Gestión de mantenimientos	HU-FLEET-008	fleet-maintenance · `GET/POST /maintenances`, `PATCH /maintenances/{id}`, `PATCH /maintenances/{id}/status`, `POST /vehicles/{id}/maintenance/complete`	`ManageMaintenanceUseCaseImplTest`, `CompleteMaintenanceUseCaseImplTest`, `ListMaintenancesUseCaseImplTest`	✅
RF3.1	Crear una reserva (con seguro opcional)	HU-BOOKING-001, HU-BOOKING-002, HU-BOOKING-003, HU-BOOKING-007	booking-reservation · `POST /reservations`, `GET /reservations/insurance-types`	`ReservationTest`, `InsuranceTypeTest`	✅ (la aceptación de términos, HU-BOOKING-007, no se verificó en el backend)
RF3.2	Consultar y modificar las propias reservas	HU-BOOKING-004, HU-BOOKING-005	booking-reservation · `GET /reservations/mine`, `GET /reservations/{id}`, `PATCH /reservations/{id}` (solo la sucursal de devolución, hasta 3 días antes: `Reservation.changeReturnBranch`)	`ReservationTest`	✅
RF3.3	Cancelar una reserva	HU-BOOKING-006	booking-reservation · `POST /reservations/{id}/cancel`; expiración automática de reservas sin pago	`ReservationTest`, `ReservationExpirationJobTest`	✅
RF8.5	Gestión administrativa de reservas y seguros	HU-BOOKING-008, HU-BOOKING-009	booking-reservation · `GET /reservations/admin`, `GET /rentals/admin`, `POST /reservations/insurance-types`	`AdminBookingControllerTest`, `InsuranceTypeTest`	✅
RF4.1	Registrar la recogida del vehículo	HU-RENTAL-001	booking-reservation · `POST /rentals/{reservationId}/pickup`	`RentalTest`	✅
RF4.2	Registrar la devolución del vehículo	HU-RENTAL-002	booking-reservation · `POST /rentals/{id}/return`	`RentalTest`	✅
RF5.1	Pago por transferencia con revisión del comprobante	HU-PAYMENT-001, HU-PAYMENT-002	payment-billing · `POST /payments`, `PATCH /payments/{id}/approve`, `PATCH /payments/{id}/reject`; booking-reservation reacciona al resultado vía `POST /internal/events`	`PaymentTest`, `PaymentUseCaseImplTest`, `PaymentEventUseCaseImplTest`	✅
RF5.2	Generar y consultar la factura	HU-PAYMENT-003, HU-PAYMENT-004	payment-billing · `GET /invoices/{reservationId}`, `GET /invoices/{reservationId}/file`	No se identificó una prueba específica de facturas	⚠️
RF8.6	Gestión de cuentas bancarias (superadministrador)	HU-PAYMENT-005	payment-billing · `GET/POST /bank-accounts`, `PATCH /bank-accounts/{id}`	No se identificó una prueba específica	⚠️
RF7.1	Notificaciones dentro de la aplicación	HU-NOTIF-001	booking-reservation · `GET /notifications`, `PATCH /notifications/{id}/read`	No se identificó una prueba específica	⚠️

RF	Descripción	Historia(s)	Implementación (servicio · endpoint)	Pruebas automatizadas	Estado
RF6.1	Ubicación de un vehículo alquilado	HU-GPS-001	telemetry-gps: sin repositorio ni servicio	—	❌ planificado
RF6.2	Historial de recorrido	HU-GPS-002	telemetry-gps: sin repositorio ni servicio	—	❌ planificado

3. Componentes transversales

Requisito	Implementación	Pruebas automatizadas
Autenticación en la entrada (JWT) y enrutamiento	api-gateway · `JwtAuthenticationFilter` y tabla de rutas en `application.properties`	`JwtAuthenticationFilterTest`
Limitación de tráfico	api-gateway · rate limiter en memoria	`InMemoryRateLimiterTest`
Trazabilidad de peticiones	api-gateway · `CorrelationIdFilter`	`CorrelationIdFilterTest`
Verificación de tokens en los servicios (RS256 vía JWKS de iam)	fleet-maintenance · adaptador de verificación de tokens	`JwksTokenVerifierAdapterTest`

4. Versiones y repositorios

Los repositorios no tienen etiquetas (tags) de versión. Se identifica el cambio por rama y último commit a la fecha de elaboración.

Repositorio	Rama	Último commit	Fecha
rtm-api-gateway	develop	`8f7cfad`	2026-10-08
rtm-iam	feat/updatePassword	`09ee3d7`	2026-10-15
rtm-fleet-maintenance	develop	`f6be34b`	2026-10-06
rtm-booking-reservation	develop	`e25f0b3`	2026-10-08
rtm-payment-billing	develop	`6cdcfd6`	2026-10-07
rtm-frontend	develop	`b307462`	2026-10-15
rtm-iam-db	develop	`25ffaf9`	2026-10-02
rtm-fleet-maintenance-db	develop	`fe861e6`	2026-10-06
rtm-booking-db	develop	`05405bd`	2026-10-07
rtm-payment-billing-db	develop	`0f67897`	2026-10-07

5. Hallazgos para el equipo

1. Cinco requisitos están implementados sin una prueba específica: facturas, cuentas bancarias, notificaciones, y parte de la recuperación de contraseña (solicitud y verificación del código).
2. `rtm-iam` está en la rama `feat/updatePassword` y no en `develop`; los cambios más recientes deben integrarse antes de la entrega.

3. Se recomienda crear una etiqueta (por ejemplo `v1.0-entrega`) en cada repositorio.
4. Las pantallas de la aplicación web que usan datos simulados (`mock-api`) no deben contarse como integradas con el backend.